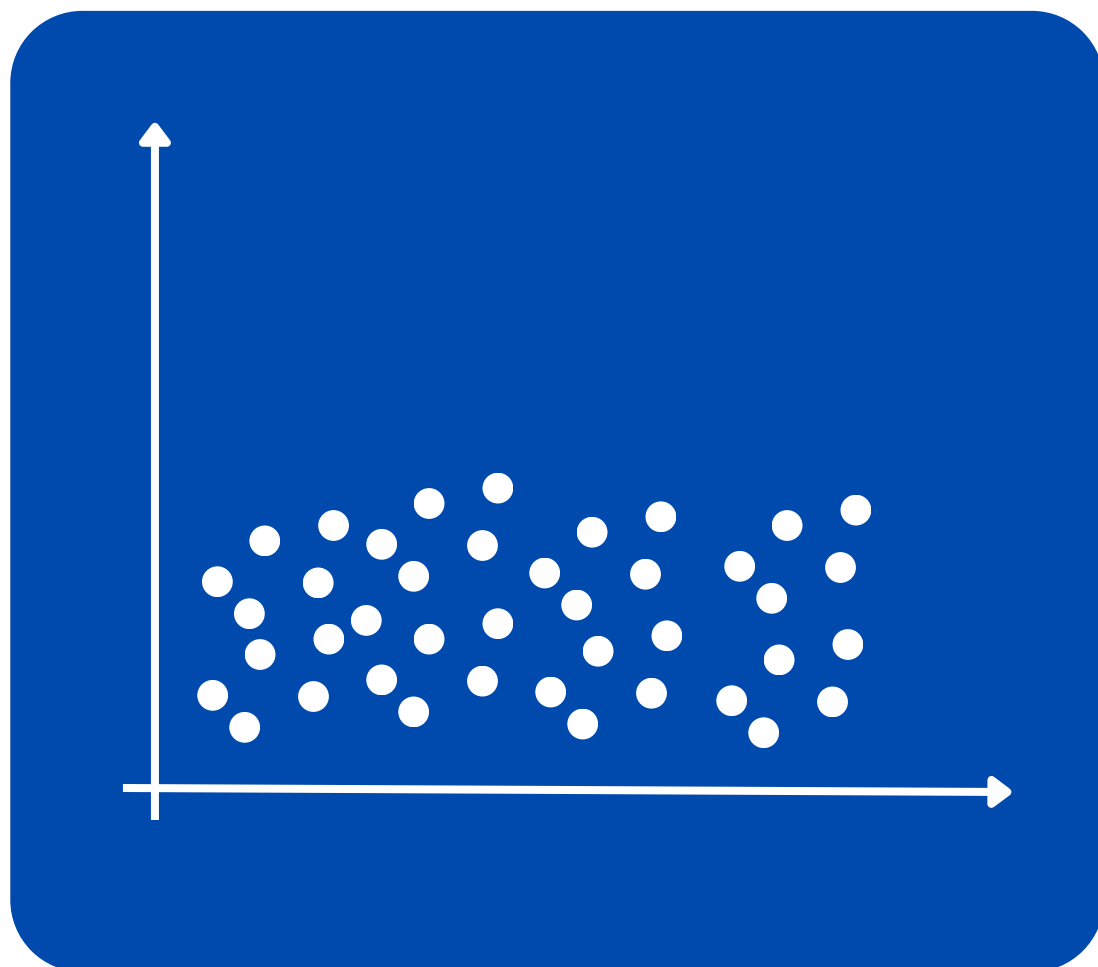
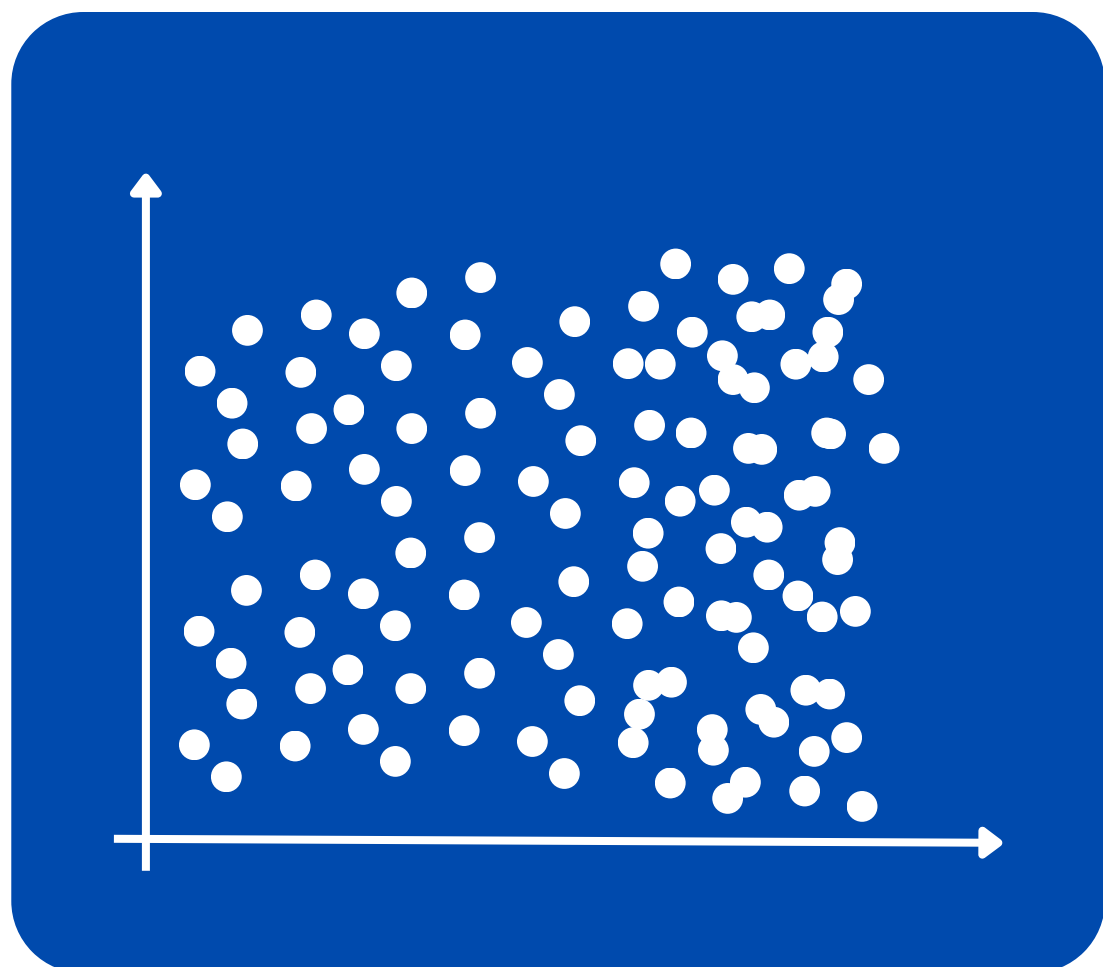
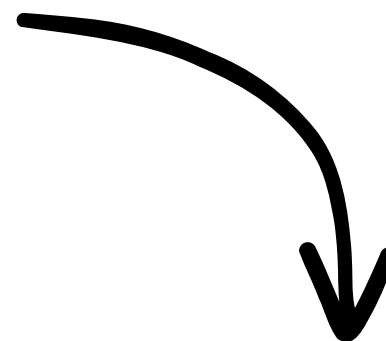


Data Normalization in Machine Learning



**After Data
Normalization**



Data normalization is essential for algorithms that are sensitive to the magnitudes of the feature values. Normalizing your data helps the algorithm in converging, i.e., to find the local/ global minimum efficiently.

Here are some common techniques for data normalization in Python using the scikit-learn library.



Min-Max Scaling

Min-Max scaling scales the features to lie in a given range, usually $[0, 1]$.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)
```

Standardization (Z-score Normalization)

Standardization rescales the features such that they have the properties of a standard normal distribution with a mean of zero and a standard deviation of one.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

Robust Scaling

Robust scaling uses the median and the interquartile range for scaling. This is robust to outliers.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
scaled_data = scaler.fit_transform(data)
```

Normalization

Normalization scales individual samples to have unit norm. This type of scaling is really important when you're using machine learning algorithms that use distance-based metrics.

```
from sklearn.preprocessing import Normalizer

scaler = Normalizer()
scaled_data = scaler.fit_transform(data)
```

Power Transformation (Box-Cox and Yeo-Johnson)

Some machine learning algorithms like linear and logistic regression assume that the variables have a Gaussian distribution. Power transformations like Box-Cox aim to make the data as "normal" as possible.

```
from sklearn.preprocessing import PowerTransformer

scaler = PowerTransformer(method='yeo-johnson')
scaled_data = scaler.fit_transform(data)
```

Quantile Transformation

Quantile transformation performs a rank mapping of your data into a uniform or Gaussian distribution.

```
from sklearn.preprocessing import QuantileTransformer

scaler = QuantileTransformer(output_distribution='normal')
scaled_data = scaler.fit_transform(data)
```